Is reimplementation of network stack a good idea or not ?

Hajime Tazaki (IIJ Research Laboratory)

Linux netdev 0x13, Prague, 2019





The research leading to these results has been supported by the EU-JAPAN initiative by the EC Horizon 2020 Work Programme (2018-2020) Grant Agreement No.814918 and Ministry of Internal Affairs and Communications "Federating IoT and cloud infrastructures to provide scalable and interoperable Smart Cities applications, by introducing novel IoT virtualization technologies (Fed4IoT)".

This talk is ...

- about my personal survey
- question: why we implement network stacks again and again ?
- Definition: network stack
  - a collection of implementations of network protocols
  - NIC driver, pkt sched, protocols (arp/ip{4,6},ndp,icmp/tcp/udp)

Network stack everywhere

- as a userspace network stack
  - mTCP, Seastar (+DPDK, netmap)
- as a container runtime
  - gVisor (netstack := Go)
- unikernel
  - lwip
  - OSv (instead of Linux guest)
  - UKL (port Linux code to unikernel specific)



https://blog.cloudflare.com/kernel-bypass/
https://www.flickr.com/photos/londonmatt/11421393074/

- **based on network-stack/kernel bypass**

Network stacks (cont'd)

	year	lang	how	API	features	original (if any)
lwip	(2001)	C	src-embedded	custom	v4,v6,ipfwd,tcp	scratch
Seastar	(2014)	C++17	static lib	custom	v4,tcp,dpdk	scratch
OSv	(2013)	C++/C	static lib	POSIX	v4,tcp	(freebsd)
gVisor	(2018)	golang	go pkg	custom	v4,v6,tcp	scratch
mTCP	(2014)	C	static lib	custom	v4,tcp,dpdk	scratch
rump	(2007)	C,asm	static/sh lib	POSIX	v4,v6,ipfwd,tcp	NetBSD
LKL	(2007?)	C,asm	static/sh lib	POSIX	v4,v6,ipfwd,tcp,dpdk	Linux
Linux	(1991)	C,asm	(kernel)	POSIX	v4,v6,ipfwd,tcp,xdp?	Linux

Network stacks (cont'd)

- some are highly optimized (perf, small footprint)
- some are feature-rich
- **how are they implemented ?**



https://www.reddit.com/r/gifs/comments/438mqv/1970s_lego_spaceship_stop_motion_build/

How to implement a network stack ?

1. full scratch
   - lwip (C), mTCP (C), Seastar (C++17), Mirage (OCaml), netstack (Go)
   - *(generally) missing features are never likely implemented*
2. port
   - OSv (FreeBSD), UKL (Linux)
   - *(generally) hard to catch up latest fixes/updates*
3. anykernel
   - Rump (NetBSD), LKL, UML (Linux)
   - *(generally) feature-rich, ease of maintenance*

is reimplementation of network stack a good idea ?

ofc not! (I hope)

Do experiments

Background: network protocol conformance

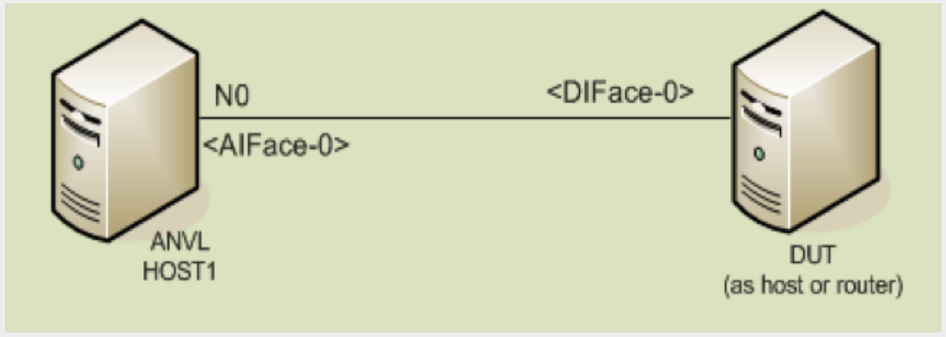
- What ?
  - measure the conformance level of implementations by a tool
- Why ?
  - measure the maturity of the network stack implementation
  - there are numbers of network stack implementations
- How ?
  - Ixia ANVL

Ixia ANVL

- IxANVL (automated network validation library)
- Validate the conformance to the standards (RFCs)
- Used to improve network stack products
- Customers: router vendors, OS vendors
  - https://www.ixiacom.com/products/ixanvl

What it looks like

- ANVL: The tester node to initiate a test
  - emulate virtual topology connected to DUT
- DUT: Device Under Test



1. ANVL: Setup (topology)
2. DUT: Configuration before a test
3. ANVL: Trigger input by packet transmission (ANVL)
4. ANVL: Wait expected response(s) from DUT

```
TEST_DESCRIPTION
 If G2 and the host identified by the internet source address
 the datagram are on the same network, a redirect message is
 to the host.
 (here gateway address must be specified)

TEST_REFERENCE
 RFC 792 p13 Redirect Message

TEST_METHOD
 SETUP: Configure DUT to add static route for host with addre
        different from host-2 via gateway RTR and outgoing in
        DIface-0
- ANVL: Send an ICMP Echo Request to DIface-0, containing:
        - IP Source Address field set to address of host-1
        - IP Destination Address field set to address of the
          address different from that of host-2
- ANVL: Listen (for upto ListenTime seconds) on DIface-0
-  DUT: Send ICMP Redirect Message

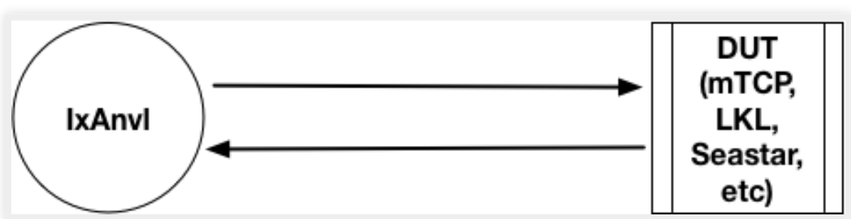
TEST_CLASSIFICATION
 MUST

TEST_TOPOLOGY
 TOPOLOGY-3
```

Test description example

Setup

- Two Linux boxes
  - Tester: Centos 6.5 (Linux 2.6.32), Ixia Anvl 9.19.9.32
  - DUT: Ubuntu 18.04 (Linux 4.15.0)
- Test suites
  - ARP (RFC826)
  - IPv4 (RFC1394, RFC1812)
  - ICMPv4 (RFC792)
  - IPv4-GW (RFC1122, RFC1812)
  - IPv6 (RFC2460, RFC2646)
  - ICMPv6 (RFC4443)
  - IPv6-NDP (RFC4861)



DUT: Various implementations

	year	lang	how	API	features	original (if any)
lwip	(2001)	C	src-embedded	custom	v4,v6,ipfwd,tcp	scratch
Seastar	(2014)	C++17	static lib	custom	v4,tcp,dpdk	scratch
OSv	(2013)	C++/C	static lib	POSIX	v4,tcp	(freebsd)
gVisor	(2018)	golang	go pkg	custom	v4,v6,tcp	scratch
mTCP	(2014)	C	static lib	custom	v4,tcp,dpdk	scratch
rump	(2007)	C,asm	static/sh lib	POSIX	v4,v6,ipfwd,tcp	NetBSD
Linux	(1991)	C,asm	(kernel)	POSIX	v4,v6,ipfwd,tcp,xdp?	Linux
LKL	(2007?)	C,asm	static/sh lib	POSIX	v4,v6,ipfwd,tcp,dpdk	Linux

Results (#pass/#total)

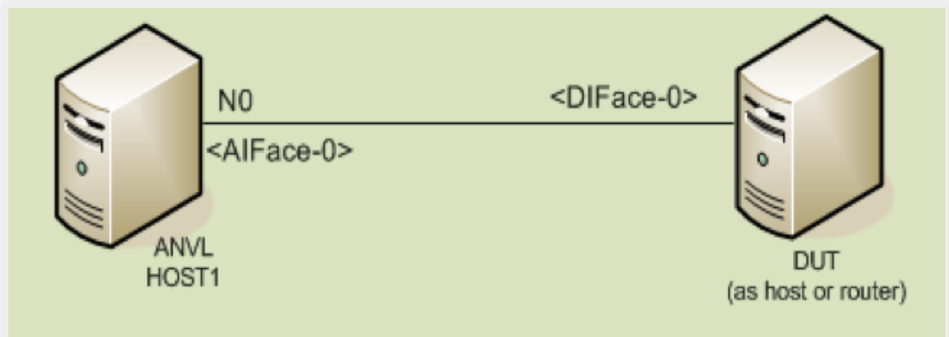
	published	ARP	IPv4	ICMPv4	IPv4-GW	IPv6	ICMPv6	IPv6-NDP
lwip	(2001)	31/52	27/68	14/32	10/18	73/75	23/45	46/150
Seastar	(2014)	32/52	**12/27**	10/22	9/17	n/a	n/a	n/a
OSv	(2013)	20/52	26/68	16/32	11/18	n/a	n/a	n/a
gVisor	(2018)	31/52	21/68	11/32	9/18	n/a	n/a	n/a
mTCP	(2014)	16/52	15/68	12/32	11/18	n/a	n/a	n/a
rump	(2007)	31/52	**13/25**	**12/26**	15/18	74/75	24/45	52/150
Linux	(1991)	47/52	41/68	22/32	17/18	72/75	26/45	78/150
LKL	(2007?)	46/52	46/68	22/32	17/18	73/75	29/45	91/150

Results contain....

- missing configuration/ability
  - arp/route entry operation (`clear arp`, etc)
  - sending ICMP req from DUT (`ping` command)
- setup failure
  - MTU isn't configured/reflected
  - no proper sysctl config
- ambiguity in specification
  - MAY/SHOULD (not MUST)
  - *impl. has options to behave blah blah ...*

Finding (incomplete implementation?)

(fail: seastar, mtcp, rump)



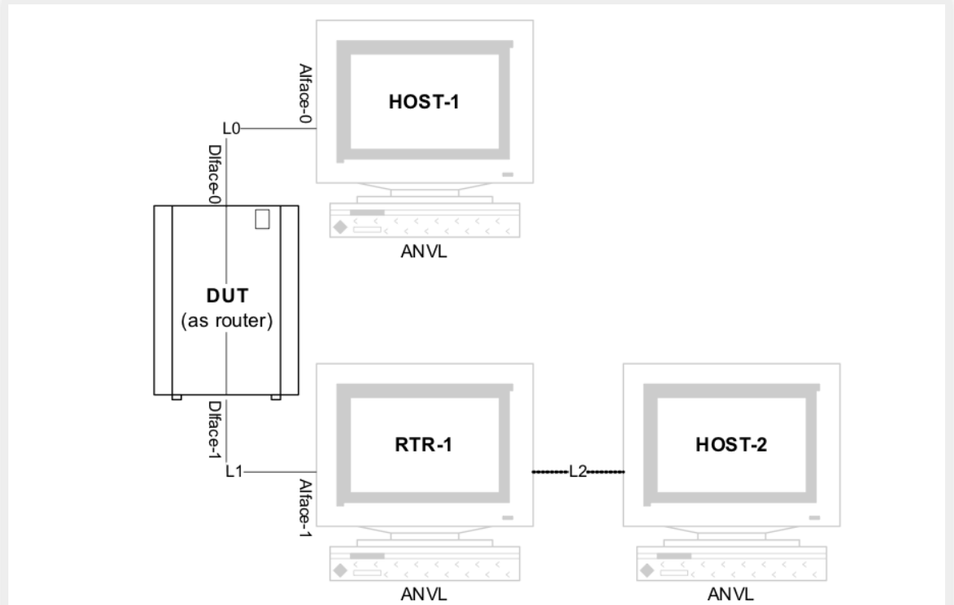
Test ARP 3.2

1. SETUP: Configure DUT to clear the dynamic entries in the ARP Cache of **DIface-0**
   - containing IP Address **HOST-1-IP**
2. ANVL: **HOST-1** Sends ARP Request to DUT through **DIface-0** containing :
   - Source IP Address set to **HOST-1-IP**
   - Destination IP Address set to **DIface-0-IP**
   - Hardware Type set to **ARP_HARDWARE_TYPE_UNKNOWN**.
3. ANVL: **HOST-1** Listens (upto) on **DIface-0**.
4. DUT: Does not send ARP Response.

3.2 When an address resolution packet is received, the receiving Ethernet module gives the packet to the Address Resolution module which goes through an algorithm similar to the following:Negative conditionals indicate an end of processing and a discarding of the packet ? Do I have the hardware type in ar$hrd? (Here ANVL is sending correct values for all the fields in the ARP Request packet except hardware type field and also ANVL is configuring DUT to clear its ARP Cache entries.The hardware type field is set to an unknown hardware type value, and ANVL expects that DUT will not send any ARP Response)

Finding (mature implementation still failed?)

(fail: linux, lkl)



Test ICMPv6-5.5

- Reason: icmpv6 code is 0 (no route to destination), should be 3 (address unreachable)
- RFC 4443 s3.1 p9 Destination Unreachable Message

  If the reason for the failure to deliver is inability to resolve the IPv6 destination address into a corresponding link address, or …, then the Code field is set to 3

Finding (nobody uses this feature ?)

(fail: all)

Test IPGW-3.4

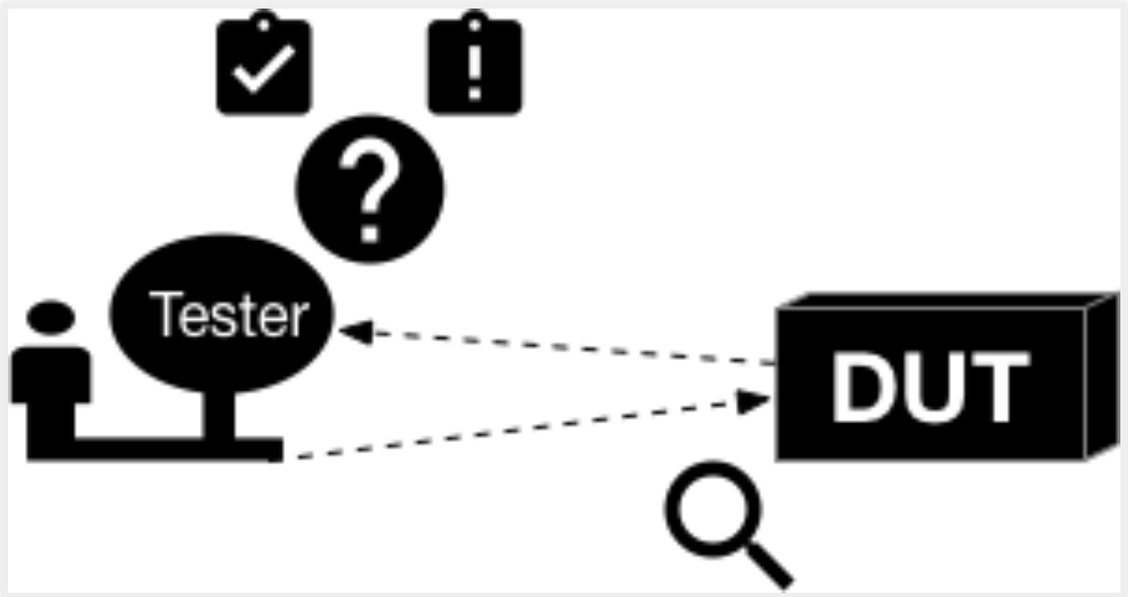
1. ANVL: Send **ICMP Address Mask Request** to DUT.
2. DUT: Send ICMP Address Mask Reply.
3. ANVL: Validate returned mask.
4. ANVL: Repeat for each configured IP interface.

Observations

- **IPv6** is 2nd-class for toy implementations
- 😟full scratch is not a good idea
  - lwip, seastar, gvisor, mTCP
  - no ip.forwarding
  - no IPv6 (except lwip)
- 😎 LKL =~ Linux
  - But they still have flaws (fragmentation, ICMP rep code, etc)
- 😢rump (NetBSD) has some errors especially in IPv4
  - 😵some tests cause panic (crash) of DUT

How Linux is mature enough ?

- some tests failed
  - returned ICMP code
  - fragment handling
  - missing features



Failed tests (Linux/LKL)

- IP-7.6: fragment packet handling incorrect ?
  - workaround: `rmmod nf_conntrack.ko`
- ICMP-2.2: fragmented packet handling
  - ICMP param problem should send
  - workaround: `rmmod nf_conntrack.ko`
- IPV6-8.1: duplicated fragmented packet handling
  - RFC 2460: failure, but in RFC8200: should be okay
- ICMPv6-5.5: "RFC 4443 (lkl/linux) icmpv6 code is 0, should be 3"
  - nhop resolution failure should be 3 (ICMPV6_ADDR_UNREACH)

Summary

- maturing a network stack is hard
- (and) choosing full-scratch implementation is tempting
- (but) using long-lived network stack is a best option